Chapter 24 | Exercise solutions with Python

Speech and Audio

This notebook solves Exercises 5–8. It traces frame starts and frequency bins on a silent waveform, locates peaks for two generated tones, and compares ways of counting transcript errors.

The examples run locally with NumPy. They use small waveforms and word lists so that the expected axes, counts, and rates can be checked by hand. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch24_speech_audio"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check the spectrogram shape

The 16 zero samples have indices 0 through 15. An eight-sample frame starting at index s needs the samples through $s + 7$. Stepping by four gives three complete frames, covering indices 0–7, 4–11, and 8–15. A fourth frame starting at 12 would need samples through index 19, so the function drops it.

Each row of the power array corresponds to a frame start. Divide its start index by $f_s = 8,000$ samples per second to get $[0, 0.0005, 0.001]$ seconds, or $[0, 0.5, 1]$ milliseconds. Each column corresponds to a frequency bin. With $N = 8$ samples per frame, the spacing is $f_s/N = 1,000$ Hz. The real FFT keeps bins 0 through 4, from 0 Hz (the constant component) to 4,000 Hz (half the sample rate). Three starts and five frequencies give a power array of shape $(3, 5)$.

The Hann window multiplies each sample by a weight. All the samples remain zero, so every Fourier coefficient is zero. The function returns raw squared Fourier magnitudes, and all $3 \times 5 = 15$ entries are exactly zero.

```
from examples.ch24_speech_audio.model import frame_power_spectrogram

start_seconds, bin_hz, power = frame_power_spectrogram(
    np.zeros(16), sample_rate=8000, frame_length=8, hop_length=4
)

np.testing.assert_array_equal(start_seconds, [0, 0.0005, 0.001])
np.testing.assert_array_equal(bin_hz, [0, 1000, 2000, 3000, 4000])
np.testing.assert_array_equal(power, np.zeros((3, 5)))

print("Frame starts (s)", start_seconds)
print("Frame starts (ms)", start_seconds * 1000)
print("Bin frequencies (Hz)", bin_hz)
print("Power shape", power.shape)
print(power)
```

OUTPUT

```
Frame starts (s) [0.     0.0005 0.001 ]
Frame starts (ms) [0.  0.5 1. ]
Bin frequencies (Hz) [   0. 1000. 2000. 3000. 4000.]
Power shape (3, 5)
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

6. Find a tone

At 8,000 samples per second, a 1,000 Hz sine repeats every eight samples, so a 16-sample frame contains two cycles. A 2,000 Hz sine repeats every four samples, giving four cycles per frame. With a hop of eight, the complete frames cover indices 0–15, 8–23, and 16–31. Their starts are 0, 1, and 2 milliseconds. Each hop is a whole number of cycles for both tones, so a tone begins at the same phase in all three frames.

Bin k measures k cycles per frame. The spacing is $8,000/16 = 500$ Hz, so the bins run from 0 to 4,000 Hz in 500 Hz steps. We expect the 1,000 Hz tone to peak in bin 2 and the 2,000 Hz tone to peak in bin 4.

The function tapers each frame with the symmetric Hann window. For a local sample index n from 0 through 15, the weight is $w[n] = (1 - \cos(2\pi n/15))/2$. The cosine makes one full turn between indices 0 and 15, giving both end samples zero weight. A real sine has a positive and a negative frequency component. When the Fourier transform tests the matching positive bin, the phase of the positive component cancels against the phase of that bin's Fourier wave. Its contributions therefore add in proportion to the sum of the window weights. The taper also spreads energy into neighboring bins. To see which bin wins after tapering, we check all nine squared magnitudes against the direct Fourier sum with the same weights.

```
from examples.ch24_speech_audio.model import frame_power_spectrogram

sample_rate = 8000
frame_length = 16
hop_length = 8
sample_indices = np.arange(32)
starts = [0, 8, 16]
offsets = np.arange(frame_length)
bins = np.arange(9)

# Write out the weights and Fourier waves for the direct sum.
window = 0.5 - 0.5 * np.cos(
    2 * np.pi * offsets / (frame_length - 1)
)
angles = -2j * np.pi * np.outer(bins, offsets) / frame_length
fourier_waves = np.exp(angles)

for tone_hz, expected_bin in [(1000, 2), (2000, 4)]:
    phase = 2 * np.pi * tone_hz * sample_indices / sample_rate
    waveform = np.sin(phase)
    start_seconds, bin_hz, power = frame_power_spectrogram(
        waveform, sample_rate, frame_length, hop_length
    )

    frames = np.stack(
        [waveform[start:start + frame_length] for start in starts]
    )
    direct_power = np.abs((frames * window) @ fourier_waves.T) ** 2
    np.testing.assert_array_equal(start_seconds, [0, 0.001, 0.002])
    np.testing.assert_array_equal(bin_hz, 500 * bins)
    np.testing.assert_allclose(
        power, direct_power, rtol=1e-12, atol=1e-12
    )

    peak_bins = power.argmax(axis=1)
    np.testing.assert_array_equal(peak_bins, [expected_bin] * 3)
    np.testing.assert_array_equal(bin_hz[peak_bins], [tone_hz] * 3)

    ranked_power = np.sort(power, axis=1)
    print(f"{tone_hz} Hz tone")
    print("  Peak bins by frame", peak_bins)
    print("  Peak frequencies (Hz)", bin_hz[peak_bins])
    print(f"  First frame peak power {ranked_power[0, -1]:.6f}")
    print(f"  First frame next-highest {ranked_power[0, -2]:.6f}")
```

OUTPUT

```
1000 Hz tone
  Peak bins by frame [2 2 2]
  Peak frequencies (Hz) [1000. 1000. 1000.]
  First frame peak power 13.974058
  First frame next-highest 4.307203
2000 Hz tone
  Peak bins by frame [4 4 4]
  Peak frequencies (Hz) [2000. 2000. 2000.]
  First frame peak power 14.062500
  First frame next-highest 4.207856
```

Using `argmax(axis=1)` picks one bin from each frame row. The peak bin is 2 in all three rows for the first tone and 4 in all three rows for the second. The first-frame comparison shows how much higher the matching bin is than the next-highest bin. The same sample pattern, with the same taper, gives the other two frames the same spectrum up to floating-point roundoff.

7. Compare clip and corpus errors

The first reference, `go home`, has two words. The hypothesis keeps `go` and drops `home`, so one deletion gives a word error rate (WER) of 1/2. The second hypothesis, `please turn left now`, matches its four-word reference exactly, so its WER is 0/4.

Reference	Hypothesis	Edits	Reference words	Clip WER
go home	go	1	2	50%
please turn left now	please turn left now	0	4	0%

Pass one list of words for each reference and hypothesis. Check the returned edit counts and denominators before combining them.

```
from examples.ch24_speech_audio.model import word_error_rate

short_clip = word_error_rate(["go", "home"], ["go"])
long_clip = word_error_rate(
    ["please", "turn", "left", "now"],
    ["please", "turn", "left", "now"],
)

assert short_clip == {
    "substitutions": 0, "deletions": 1, "insertions": 0,
    "edits": 1, "reference_words": 2, "wer": 0.5,
}
assert long_clip == {
    "substitutions": 0, "deletions": 0, "insertions": 0,
    "edits": 0, "reference_words": 4, "wer": 0.0,
}

labeled_clips = [("Short clip", short_clip), ("Long clip", long_clip)]
for label, result in labeled_clips:
    edits = result["edits"]
    words = result["reference_words"]
    rate = result["wer"]
    print(f"{label}  {edits}/{words} = {rate:.0%}")
```

```
OUTPUT
Short clip  1/2 = 50%
Long clip  0/4 = 0%
```

Pooled WER adds the edit counts and divides by the total number of reference words. The clip mean, also called a macro average, takes the arithmetic mean of the two clip rates.

$$WER_{pooled} = \frac{1+0}{2+4} = \frac{1}{6}, \qquad WER_{macro} = \frac{1/2 + 0/4}{2} = \frac{1}{4}.$$

The clip mean gives each clip half the weight. Pooling is equivalent here to weighting the clip rates by their reference lengths. The short clip accounts for 2/6 of the reference words and the long clip accounts for 4/6. The error occurs in the shorter clip, so pooled WER is about 16.67% and mean clip WER is 25%.

```
clips = [short_clip, long_clip]
total_edits = sum(clip["edits"] for clip in clips)
total_words = sum(clip["reference_words"] for clip in clips)
pooled_wer = total_edits / total_words
mean_clip_wer = sum(clip["wer"] for clip in clips) / len(clips)

assert (total_edits, total_words) == (1, 6)
assert pooled_wer == 1 / 6
assert mean_clip_wer == 1 / 4

print(f"Pooled WER     {pooled_wer:.2%}")
print(f"Mean clip WER  {mean_clip_wer:.2%}")
```

OUTPUT

```
Pooled WER     16.67%
Mean clip WER  25.00%
```

8. Audit a tie

The reference is `a b` and the hypothesis is `b a`. A single insertion or deletion changes the length, and a single substitution changes only one position. The words differ at both positions, so at least two edits are needed. In the table, each column of an alignment pairs a reference entry with a hypothesis entry. A dash marks a gap.

Alignment	Row	First	Second	Third
A	Reference	a	b	
A	Hypothesis	b	a	
A	Operation	Substitute	Substitute	
B	Reference	a	b	–
B	Hypothesis	–	b	a
B	Operation	Delete	Match	Insert

Alignment A substitutes both words. Alignment B deletes the first `a`, keeps the `b`, and inserts an `a` at the end. Both use exactly two edits, so both are optimal. Their counts (S, D, I) are $(2, 0, 0)$ and $(0, 1, 1)$. In either alignment the WER is $2/2 = 100\%$.

The function compares candidate final steps for each pair of prefixes. For the full pair `a b` and `b a`, all three steps have the same cost.

Final step	Prefixes before the step	Cost
Substitute final `b` with `a`	`a` and `b`	$1 + 1 = 2$
Delete final `b`	`a` and `b a`	$1 + 1 = 2$
Insert final `a`	`a b` and `b`	$1 + 1 = 2$

The code tests the diagonal first, then deletion, then insertion, and keeps the first minimum. The diagonal pairs the last reference word with the last hypothesis word and charges a substitution because they differ. Its predecessor aligns `a` with `b` by substitution, giving two substitutions in total. The implementation carries those counts forward while filling the table, so it can report them without storing the alignment.

```
from examples.ch24_speech_audio.model import word_error_rate

tied = word_error_rate(["a", "b"], ["b", "a"])
assert tied == {
    "substitutions": 2, "deletions": 0, "insertions": 0,
    "edits": 2, "reference_words": 2, "wer": 1.0,
}

print(f"Substitutions    {tied['substitutions']}")
print(f"Deletions        {tied['deletions']}")
print(f"Insertions       {tied['insertions']}")
print(f"Total edits      {tied['edits']}")
print(f"Reference words  {tied['reference_words']}")
print(f"WER              {tied['wer']:.0%}")
```

```
OUTPUT
Substitutions    2
Deletions        0
Insertions       0
Total edits      2
Reference words  2
WER              100%
```